

Angle constrained Paths

Saeed Asaeedi (✉ asaeeedi@kashanu.ac.ir)

University of Kashan

Mahsa Soheil Shamaee (✉ soheilshamaee@kashanu.ac.ir)

University of Kashan

Research Article

Keywords:

DOI: <https://doi.org/>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Angle constrained Paths

Saeed Asaeedi^{1*} and Mahsa Soheil Shamaee¹

¹Department of Computer Science, Faculty of Mathematical Sciences, University of Kashan, Kashan 87317-53153, I. R. Iran.

*Corresponding author(s). E-mail(s): asaeedi@kashanu.ac.ir;
Contributing authors: soheilshamaee@kashanu.ac.ir;

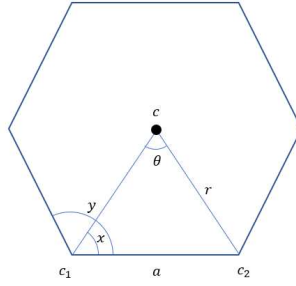
Abstract

The traveling salesman problem (TSP) is a well-known and most widely studied problem in combinatorial optimization. The Dubins TSP is a variant of TSP for curvature-constrained vehicles. In this paper, another variant of TSP, called angle-constrained TSP, is considered for vehicles that can either move straight or turn, i.e., no voluntary curve is possible. The vehicles are only allowed to make on-the-spot turns and straight motions. We impose turn angle constraint on vehicles. Hence, some additional arbitrary points, called stop points, are added to reach an unreachable target. Here, computing the minimum number of stop points walking on the optimal TSP path is presented as the solution of angle-constrained TSP. In addition, we consider two other problems: computing an angle-constrained path on the vertices with the minimum number of stop points and computing an angle-constrained path with the minimum perimeter. In both problems, there is no need to stay on the optimal TSP path. Here, some algorithms are developed to solve these three problems and are examined on some datasets. These algorithms can be used for the path planning of robots with turn angle constraints.

Keywords: Angle-Constrained TSP, Angle-Constrained MSP, Angle-Constrained MPP, Dubins TSP, TSP, Computational Geometry

1 Introduction

TSP is a well-known NP-hard problem with many applications in the fields of transportation [1], robot motion planning [2], wireless sensor networks [3],

2 *Angle constrained Paths***Fig. 1** Notations of a regular polygon

path planning [4, 5], and GIS [6]. There are many exact [7–9] and heuristic [7, 10–13] algorithms to solve this problem. The traveling salesman problem for curvature-constrained vehicles is known as the Dubins TSP [14]. A lot of studies have been conducted on Dubins TSP [15–18] using vehicles that can turn while moving.

In this paper, the vehicles that can either move straight or turn, i.e., no voluntary curve is possible, are considered. Another constraint on these vehicles is the turn angle constraint. Sujit et al. [19] designed a route path for angle-constrained terrain using an unmanned aerial vehicle. Bill Triggs [20] presented an introduction to motion planning for vehicles with rolling constraints such as Dubins vehicles and cars with trailers.

The rest of the paper is as follows. The notations are defined in section 2. In section 3 angle-constrained TSP, angle-constrained MSP, and angle-constrained MPP are defined as the problems of computing the minimum number of stop points walking on the optimal TSP path, computing a path with the minimum number of stop points and computing a path with the minimum perimeter, respectively. Section 4 concludes the paper by highlighting its achievements.

2 Preliminaries

Let $S = \{s_1, s_2, \dots, s_n\}$ be a set of vertices in the plane and $T = \langle t_1, t_2, \dots, t_n, t_1 \rangle$ the optimal TSP path on S . Considering an angle α between 0 and π , we compute a path, denoted by $CT = \langle x_1, x_2, \dots, x_1 \rangle$, such that all turn angles of CT are less than or equal to α . Each point x_i either belongs to S or stop points. We call the CT an angle-constrained path. The notations that are used in the rest of the paper are shown in Table 1.

Let P be a regular polygon with N vertices as shown in Fig. 1. The side length, center, circumradius, and interior angle of P are denoted by a , c , r , and y , respectively. The base angles of triangle $\triangle c_1 c_2 c$ are shown by x . Clearly $x = \frac{y}{2}$.

Table 1 Notations of symbols

Notation	Description
S	a set of vertices in the plane
n	cardinality of S
T	optimal TSP path on S
t_i	i th vertex visited by T ($1 \leq i \leq n$)
DT	Dubins path on S
SP	set of stop points added for rerouting on sharp angles
m	cardinality of SP
CT	angle-constrained path on $S \cup SP$
x_i	i th vertex visited by CT ($1 \leq i \leq n + m$)
e_i	i th edge of T , ie, $e_i = \overline{t_i t_{i+1}}$ ($1 \leq i \leq n - 1$)
e_n	last edge of T , ie, $e_n = \overline{t_n t_1}$
$\hat{t}_i$	turn angle on t_i . ie, $\hat{t}_i = \pi - \widehat{t_{i-1} t_i t_{i+1}}$ ($2 \leq i \leq n - 1$)
$\hat{t}_n$	turn angle on t_n . ie, $\hat{t}_n = \pi - \widehat{t_{n-1} t_n t_1}$
α	turn angle constraint ($0 \leq \alpha \leq \pi$)
angle-constrained path	a path with turn angles less than or equal to α
P	a regular polygon
N	number of vertices of P
a, c, r and y	side length, center, circumradius and interior angle of P , respectively
P_i	regular polygon on vertex t_i

3 Angle-constrained paths

In this section, we consider three problems to compute different angle-constrained paths on S . In section 3.1, the problem of computing a path with the minimum number of stop points is discussed to visit all vertices of S in order of vertices of T . Here, we call this problem the "angle-constrained TSP". In section 3.2, an angle-constrained path is computed to meet all vertices of S with the minimum number of stop points. The problem of computing such a path is named "angle-constrained MSP". In section 3.3, "angle-constrained MPP" is considered as the problem of computing an angle-constrained path on S with the minimum perimeter.

3.1 Angle-constrained TSP

Here, the CT is computed on a set S of vertices adding stop points SP to S such that the order of visited vertices of S in CT is the same as that of T . We initialize the path with T and add stop points facing sharp angles. Traveling on the path T , stop points will be added where a vertex t_i is met such that $\hat{t}_i > \alpha$. Fig. 2 illustrates an example of computed CT on a set of 50 vertices under the turn angle constraint $\alpha = 45^\circ$. The goal is to place the minimum number of stop points in a way that all visited vertices of S be in the same order as T . The presented algorithm of computing CT in this section satisfies the goal. The angle-constrained TSP is introduced in the following definition:

Definition 1 Let S be a set of vertices in the plane and α an angle between 0 and π . The *angle-constrained TSP* is the problem of finding a path $CT = \langle x_1, x_2, \dots, x_{n+m}, x_1 \rangle$ with the minimum number of vertices such that $\hat{x}_i \leq \alpha$ for all

4 Angle constrained Paths

$i \in \{1, 2, \dots, n+m\}$ and CT crosses all the vertices of S in the order of vertices of T . We named CT , the angle-constrained TSP path.

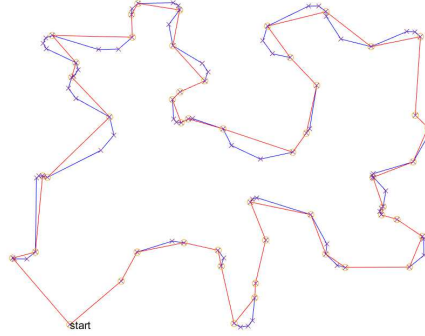


Fig. 2 Stop points are depicted by $\times$ and T and CT are shown by the red and blue lines, respectively

let $N = \lceil \frac{2\pi}{\alpha} \rceil$ and $\langle t_{i-1}, t_i, t_{i+1} \rangle$ be a subpath of T such that $\beta = \hat{t}_i > \alpha$. A regular polygon P_i with N vertices is said to be on vertex t_i if both t_i and t_{i+1} are on P_i and t_i is placed on the vertices of P_i (see Fig. 3). Algorithm 1 computes the circumradius of P_i based on L , θ_1 and θ_2 such that L is the length of e_i , θ_1 is the angle subtended by e_i at c and θ_2 is the angle subtended by r at t_{i+1} .

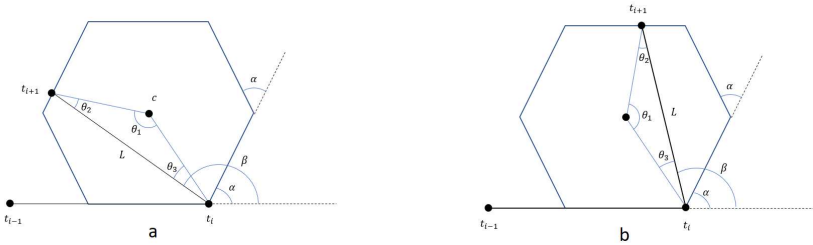


Fig. 3 The regular polygon P_i is on vertex t_i , a) number of stop points is equal to or more than $3 = \frac{N}{2}$, b) number of stop points is less than $3 = \frac{N}{2}$

The input parameters of algorithm 1 are N as the number of vertices of P_i , α as the exterior angle of P_i , β as the turn angle of $\hat{t}_i$ and (x_2, y_2) and (x_3, y_3) as the coordinates of t_i and t_{i+1} , respectively. The procedure *getStopPointsCount* computes the number of stop points needed to reach t_{i+1} under the turn angle constraint. θ_1 and θ_3 are computed in different ways based on the number of stop points, as shown in Fig. 3. So, the procedures

Algorithm 1 Computing circumradius of the regular polygon**Require:** $N, \alpha, \beta, x_2, y_2, x_3, y_3$ **Ensure:** r as circumradius of regular polygon $StopPointsCount \leftarrow getStopPointsCount(N, \beta, \alpha)$ $\theta_1 \leftarrow computeTheta1(N, StopPointsCount)$ $\theta_3 \leftarrow computeTheta3(N, \beta, \alpha, StopPointsCount)$ $\theta_2 \leftarrow \pi - (\theta_1 + \theta_3)$ $L \leftarrow \sqrt{(x_3 - x_2)^2 + (y_3 - y_2)^2}$ $r \leftarrow L * \sin \theta_2 / \sin \theta_1$ **procedure** GETSTOPPOINTSCount(N, β, α) $j \leftarrow \frac{\pi - \alpha}{N - 2} \quad \triangleright \text{angle subtended by an edge of regular polygon at } t_i$ $def \leftarrow \beta - \alpha$ $count \leftarrow \frac{def}{j} \quad \triangleright \text{count is a real number}$ **return** $count$ **end procedure****procedure** COMPUTETHETA1($N, StopPointsCount$) $k \leftarrow 2\pi / N$ **if** $StopPointsCount < \frac{N}{2}$ **then** $\theta_1 \leftarrow StopPointsCount * k$ **else** $\theta_1 \leftarrow (N - StopPointsCount) * k$ **end if****return** θ_1 **end procedure****procedure** COMPUTETHETA3($N, \beta, \alpha, StopPointsCount$) $x \leftarrow \frac{(N-2)\pi}{2N} \quad \triangleright \text{half of interior angle of regular polygon}$ **if** $StopPointsCount < \frac{N}{2}$ **then** $\theta_3 \leftarrow x - (\beta - \alpha)$ **else** $\theta_3 \leftarrow \beta - \alpha - x$ **end if****return** θ_3 **end procedure**

$computeTheta1$ and $computeTheta3$ use different methods to compute θ_1 and θ_3 .

In the following, a theorem is presented to find a path to visit an unreachable vertex, using stop points and under the turn angle constraint. Then, we try to minimize the number of stop points.

Theorem 1 *let $\langle t_{i-1}, t_i, t_{i+1} \rangle$ be a subpath of T such that $\beta = \hat{t}_i > \alpha$. There exists a new path $\langle t_{i-1}, t_i, c_1, c_2, \dots, c_k, t_{i+1} \rangle$ such that $\hat{t}_i \leq \alpha$ and $\hat{c}_j \leq \alpha$ for all $j \in \{1, 2, \dots, k\}$.*

6 *Angle constrained Paths*

Proof Since the turn angle β from t_i to t_{i+1} is more than the turn angle constraint α , a sequence of turn and straight movements is needed to reach t_{i+1} . As shown in Fig. 4, the vertices of this sequence are denoted by stop points c_j . We place these stop points on the vertices of the regular polygon P_i which is on vertex t_i . If the interior angle of P_i be $\pi - \alpha$, the turn angle in each vertex of P_i will be α . Therefore, placing the stop points $c_1, c_2, \dots, c_k$ on the vertices of P_i leads to $\hat{c}_j \leq \alpha$ for all $j \in \{1, 2, \dots, k\}$.

The circumradius r is computed by algorithm 1. So, the side length of P_i is $a = 2r \sin(\frac{\pi}{N})$ such that N is the number of vertices of P_i and $N \geq \lceil \frac{2\pi}{\alpha} \rceil$. The condition $N \geq \lceil \frac{2\pi}{\alpha} \rceil$ yields $\hat{c}_j \leq \alpha$ for all $j \in \{1, 2, \dots, k\}$ and $a = 2r \sin(\frac{\pi}{N})$ results in reaching t_{i+1} after c_k . The value of k depends on α and β and not on the length of $L = \overline{t_i t_{i+1}}$. This length affects the side length, circumradius, and inradius of the regular polygon.

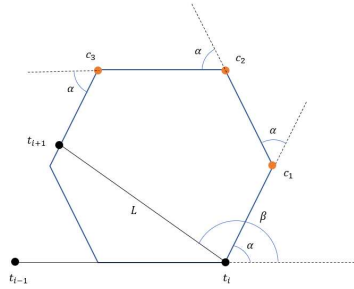


Fig. 4 The regular polygon P_i is on vertex t_i and c_1, c_2 , and c_3 are stop points

Reducing the number of stop points:

Let h be the exterior angle of P_i . We know that $h = \pi - \frac{(N-2)\pi}{N}$. Satisfying $h \leq \alpha$ yields us $N \geq \frac{2\pi}{\alpha}$. Based on the procedure *getStopPointsCount* in algorithm 1, for $N = \lceil \frac{2\pi}{\alpha} \rceil$, we can use more than $\lfloor \frac{(\beta-\alpha)*(N-2)}{\pi-\alpha} \rfloor$ stop points to guarantee the turn angle constraint. Using any value less than $\lfloor \frac{(\beta-\alpha)*(N-2)}{\pi-\alpha} \rfloor$ for k as the number of stop points, either leads to $\hat{t}_i > \alpha$ or $\hat{c}_j > \alpha$ for any $j \in \{1, 2, \dots, k\}$ or changes the value of $\hat{t}_{i+1}$. In the cases of $\hat{t}_i > \alpha$ and $\hat{c}_j > \alpha$, the turn angle constraint is violated. But we can reduce the number of stop points without violating the turn angle constraint, by changing the value of $\hat{t}_{i+1}$.

In Fig. 5, the new path $\langle t_{i-1}, t_i, c'_1, c'_2, t_{i+1} \rangle$ satisfies the angle constraint and has less stop points than $\langle t_{i-1}, t_i, c_1, c_2, c_3, t_{i+1} \rangle$. Moreover, the turn angle $\hat{t}_i$ is not changed in the new path, and we have $\hat{c}'_1 = \hat{c}_1$ and $\hat{c}'_2 = \hat{c}_2$.

Traveling on the path of stop points, starting with t_i , while the direction of travel is not towards t_{i+1} , we need to use stop points to turn to t_{i+1} . As soon as the direction of travel is towards t_{i+1} , we can skip the remainder stop points. Hence, The number of necessary stop points to reach t_{i+1} is equal to the number of edges that were not in the direction of reaching t_{i+1} . The edge

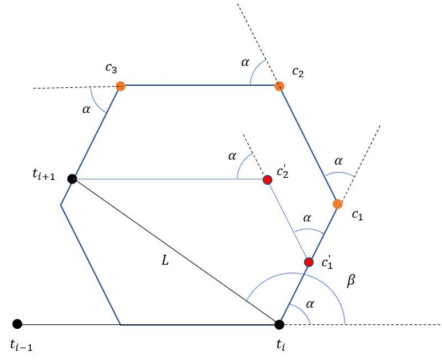


Fig. 5 Reducing the stop points from $\{c_1, c_2, c_3\}$ to $\{c'_1, c'_2\}$

$\overrightarrow{c_i c_{i+1}}$ is in the direction of reaching t_{i+1} if and only if the cross product of $\overrightarrow{c_i c_{i+1}}$ and $\overrightarrow{t_i t_{i+1}}$ is negative. Algorithm 2 computes the number of necessary stop points to reach t_{i+1} .

Algorithm 2 Computing the number of necessary stop points

Require: regular polygon P_i , t_i, t_{i+1}

Ensure: N_i as the number of necessary stop points to reach t_{i+1} from t_i

$n \leftarrow$ number of vertices of P_i

$N_i \leftarrow 0$

for $j=1$ **to** $j=n-1$ **do**

$\triangleright$ Assume that c_0 is t_i

if $|\overrightarrow{c_j c_{j+1}} \times \overrightarrow{t_i t_{i+1}}| \geq 0$ **then**

$N_i \leftarrow N_i + 1$

else

Break

end if

end for

Let e_f be the first edge in P_i that is in the direction of reaching t_{i+1} , i.e. $|\overrightarrow{c_f c_{f+1}} \times \overrightarrow{t_i t_{i+1}}| < 0$. Algorithm 2 computes the number of necessary stop points. But it needs to relocate the stop points from c_i to c'_i to reach the vertex t_{i+1} after the last turn. Consider a line parallel to e_f from the point t_{i+1} . This parallel line divides the regular polygon P_i into a certain ratio. By drawing a new regular polygon with the same ratio smaller than P_i , the new position of the stop points is computed. Fig. 6 shows two examples of relocating the stop points based on their ratio. Algorithm 3 computes the required stop points to reach t_{i+1} from t_i .

After that, we use regular polygons P'_i on each sharp turn angle $\hat{t}_i$ to compute CT as the angle-constrained TSP path. This method is described in algorithm 4. In this algorithm, we first initialize CT with the optimal TSP path T . Then on each sharp angle t_i , i.e. $\hat{t}_i > \alpha$, the stop points SP_i are added

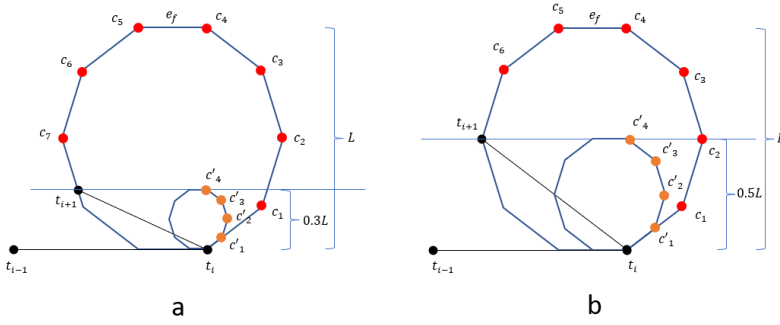


Fig. 6 Relocating stop points, a) ratio is 0.3, b) ratio is 0.5

Algorithm 3 Computing the stop points to reach t_{i+1} from t_i

Require: $N, \alpha, \beta, t_i, t_{i+1}$

Ensure: SP_i as the required stop points to reach t_{i+1}

$P_i \leftarrow$ the regular polygon on vertex t_i computed by algorithm 1 using the parameters $N, \alpha, \beta, x(t_i), y(t_i), x(t_{i+1}), y(t_{i+1})$

$N_i \leftarrow$ the number of necessary stop points to reach t_{i+1} computed by algorithm 2 using the parameters P_i, t_i, t_{i+1}

$e_f \leftarrow (N_i + 1)$ 'th edges of P_i

$L \leftarrow$ a line parallel to e_f and crossing t_{i+1}

$rto \leftarrow$ the ratio that the polygon P_i is divided in two parts using L

$a \leftarrow$ the side length of P_i

$a' \leftarrow a \times rto$

$P'_i \leftarrow$ the regular polygon on vertex t_i with side length a'

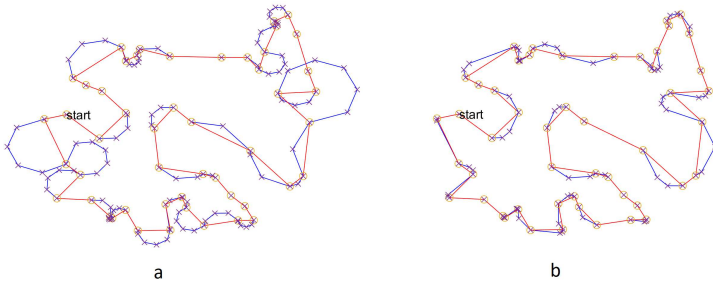
$SP_i \leftarrow$ the first N_i vertices of P'_i

to CT using algorithm 3. Adding stop points affects the next turn angle $\hat{t}_{i+1}$. We recompute the turn angle $\hat{t}_{i+1}$ and continue the algorithm to the next sharp angle. The algorithm will be finished when the start vertex is reached.

Remark 1 The procedure of reducing the stop points decreases both the number of stop points and the length of the traveled path. Fig. 7 compares the angle-constrained paths with and without reducing stop points on a set of vertices for $\alpha = 45^\circ$.

The results of algorithm 4 on a set of 50 vertices are illustrated in Fig. 8 for different values of α . As shown in Fig. 8, for small values of α the turn constraint leads to semi-curved paths, and for large values of α more freedom in rotation leads to near-optimal TSP path.

Based on our knowledge, the closest concept to ours is Dubins TSP [14]. We compare our results with the Dubins paths on datasets of 10, 20, ..., and 50 points under the turn angle constraints of 20, 30, ..., and 100 degrees.

Algorithm 4 Computing CT as the angle-constrained TSP path**Require:** S, α **Ensure:** CT as a solution of angle-constrained TSP on S $n \leftarrow \text{size of } S$ $T \leftarrow \text{optimal TSP path on } S$ $CT \leftarrow T$ $\triangleright$ initialize CT **for** $i=2$ **To** $i=n$ **do** **if** $\hat{t}_i > \alpha$ **then** $SP_i \leftarrow \text{computed stop points by algorithm 3 using the parameters } t_i$ and t_{i+1} remove edge e_i from CT and add SP_i to CT between t_i and t_{i+1} update $\hat{t}_{i+1}$ as $\pi - \widehat{c_k t_{i+1} t_{i+2}}$ **end if****end for****Fig. 7** Angle-constrained path: a) without reducing stop points, b) with reducing stop points

We compute T , DT , and CT as the optimal TSP, Dubins TSP, and angle-constrained TSP paths on each dataset, respectively. The radius of curvature in Dubins TSP corresponds to the value of the turn angle constraint.

The cost of Dubins TSP and angle-constrained TSP paths on a dataset are the ratio given by the perimeter of DT and CT , respectively divided by the perimeter of T . Fig. 9 compares the costs of Dubins TSP with those of angle-constrained TSP on mentioned datasets.

3.2 Angle-constrained MSP

In section 3.1, we find an angle-constrained path of CT , on a set of vertices S , with the minimum number of stop points traveling on T . The order of visited vertices of S in CT is the same as that of T . But by changing the order of visiting vertices in CT , we can reduce the number of stop points. Fig. 10 depicts an example of vertices of S such that the computed angle-constrained path on S has fewer stop points than that of the angle-constrained TSP path. The problem of finding a path with the minimum number of stop points, angle-constrained MSP, is defined as follows:

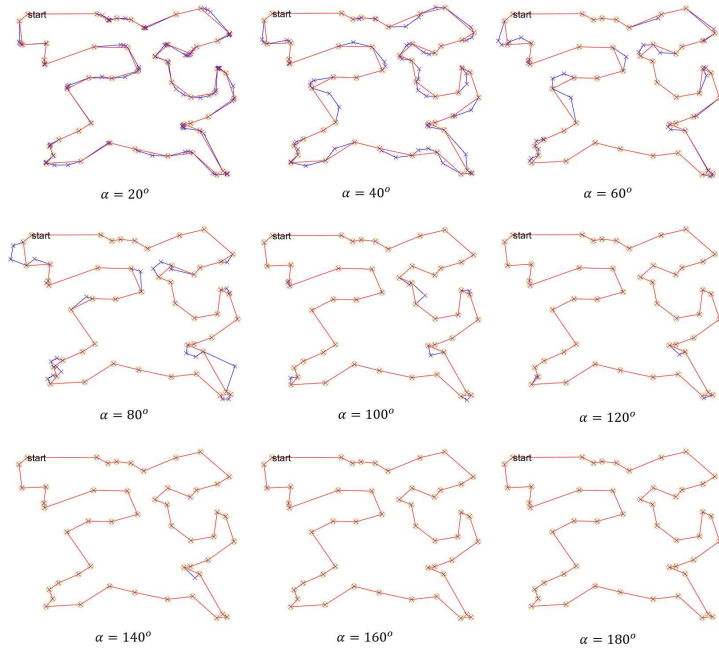


Fig. 8 The angle-constrained TSP paths on a set of vertices for different values of α

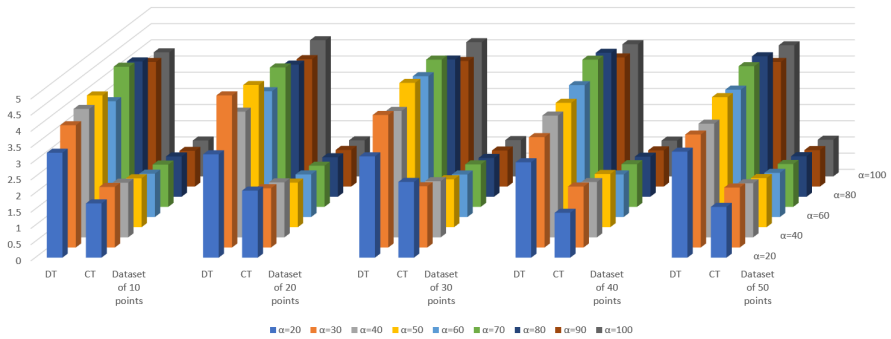


Fig. 9 The costs of angle-constrained TSP path, CT , and Dubins TSP path, DT , on datasets of 10, 20, ..., 50 points under the turn angle constraints of 20, 30, ..., 100 degrees

Definition 2 Let S be a set of vertices in the plane and α an angle between 0 and π . The *angle-constrained MSP* is the problem of finding a path $CT = \langle x_1, x_2, \dots, x_{n+m}, x_1 \rangle$ with the minimum number of vertices such that $\hat{x}_i \leq \alpha$ for all $i \in \{1, 2, \dots, n+m\}$ and $S \subseteq \{x_1, x_2, \dots, x_{n+m}\}$. We named CT , the angle-constrained MSP path.

In Fig. 10. b, we are not forced to stay on the path T . Hence, we are looking for a path that has the minimum number of stop points and satisfies

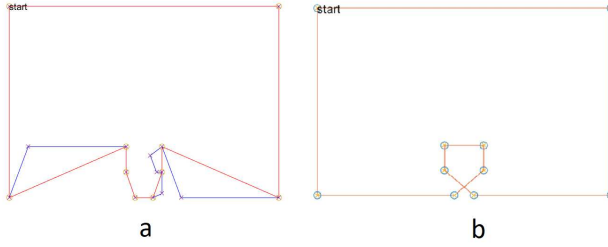


Fig. 10 a) TSP path (T) and angle-constrained TSP path (CT) with stop points, b) an angle-constrained path without stop points for $\alpha = 90^\circ$

the angular constraint. In case that there is at least one path satisfying the angular constraint, the number of required stop points is 0. Otherwise, we have to use stop points to visit all vertices. We first define *visible vertex* from an edge and *visible graph* and then find minimum number of stop points to visit all vertices.

Definition 3 A vertex s_k is said to be a *visible vertex* from the edge $\overline{s_i s_j}$ if the turn angle $\widehat{s_i s_j s_k}$ is less than or equal to α .

Definition 4 A directed graph $G = (V, E)$ is a *visible graph* such that $V = \{(s_i, s_j) : i, j \in \{1, 2, \dots, n\}, i \neq j\} \cup \{(s_1, s_1), (s_{n+1}, s_{n+1})\}$ and a directed edge from (s_i, s_j) to (s_j, s_k) belongs to E if and only if the vertex s_k is visible from the edge $\overline{s_i s_j}$. In addition, we add directed edges from (s_1, s_1) to (s_1, s_k) for all $k \in \{1, 2, \dots, n\}$ and from (s_i, s_1) to (s_{n+1}, s_{n+1}) for all $i \in \{1, 2, \dots, n\}$ to E .

We denote the vertices of G by $v_{i,j}$ and the edges of G by $e_{i,j,k}$ for all $i \neq j \neq k \in \{1, 2, \dots, n\}$. Using these definitions, we reduce our problem to the problem of finding a path without angle constraint. A path from $v_{1,1}$ to $v_{n+1,n+1}$ on G corresponds to an angle-constrained path CT on S from s_1 to s_1 . On the other hand, if there is not any path from $v_{1,1}$ to $v_{n+1,n+1}$ on G , then there is not any angle-constrained path to visit all vertices of S without stop points. Hence, to find the minimum number of stop points, we consider all missing edges of each path from $v_{1,1}$ to $v_{n+1,n+1}$ on G .

Based on the presented algorithms in section 3.1, the minimum stop points for each edge $e_{i,j,k}$ can be computed on the preprocessing phase. It is equal to the required stop points for traveling starting from s_i and hitting s_j and then turning to s_k . Therefore, based on the sum of the required stop points of missing edges, the optimal path with the minimum number of stop points can be computed. Algorithm 5 computes the visible graph on S and algorithm 6 finds the mentioned optimal path as an angle-constrained MSP path.

Since computing all possible paths between two vertices in the graph is NP-complete, we compute angle-constrained MSP paths for the set of 8, 9 and 10 vertices and $\alpha = 20^\circ$, $\alpha = 45^\circ$ and $\alpha = 90^\circ$. Fig. 11 shows these

Algorithm 5 Computing the visible graph

Require: S, α **Ensure:** G as the visible graph on S

```

 $n \leftarrow \text{size of } S$ 
for  $i=1$  to  $i=n$  do
  for  $j=1$  to  $j=n$  do
    for  $k=1$  to  $k=n$  do
      if  $i \neq j \neq k$  and  $s_k$  is visible from the edge  $\overline{s_i s_j}$  based on  $\alpha$  then
         $Visible_{i,j} \leftarrow k$ 
      end if
    end for
  end for
end for
 $G = (V, E) \leftarrow$  A directed graph
Add  $v_{1,1}$  to  $V$ 
Add  $v_{n+1,n+1}$  to  $V$ 
for  $i=1$  to  $i=n$  do
  for  $j=1$  to  $j=n$  do
    if  $i \neq j$  then
      Add  $v_{i,j}$  to  $V$ 
    end if
  end for
end for
for  $i=1$  to  $i=n$  do
  for  $j=1$  to  $j=n$  do
    for  $k=1$  to  $k=n$  do
      if  $i \neq j \neq k$  and  $k \in Visible_{i,j}$  then
        Add  $e_{i,j,k} = (v_{i,j}, v_{j,k})$  to  $E$ 
      end if
    end for
  end for
end for
for  $k=2$  to  $k=n$  do
  Add  $e_{1,1,k} = (v_{1,1}, v_{1,k})$  to  $E$ 
end for
for  $i=2$  to  $i=n$  do
  Add  $e_{i,1,n+1} = (v_{i,1}, v_{n+1,n+1})$  to  $E$ 
end for

```

angle-constrained paths. More over, Fig. 10.b is computed by the presented algorithm.

Algorithm 6 Computing angle-constrained MSP path**Require:** S, α **Ensure:** CT as a solution of angle-constrained MSP on S $n \leftarrow \text{size of } S$ $G \leftarrow$ visible graph computed by algorithm 5 using the parameters S and α $\text{minPath} \leftarrow$ find a path from $v_{1,1}$ to $v_{n+1,n+1}$ in G $\triangleright$ If there exist multiple paths, select the path with the minimum perimeter.**if** minPath is not null **then** $\triangleright$ No stop point is needed display minPath

Exit

end if $\text{ind} \leftarrow 1$ **for all** $\text{path} \in$ all possible paths from $v_{1,1}$ to $v_{n+1,n+1}$ **do** $\text{missedEdges} \leftarrow$ all edges that are in path and are not in G . $\text{StopPoints}[\text{ind}] \leftarrow$ all required stop points for missedEdges . $\triangleright$

required stop points can be computed by algorithm 3 in preprocess phase.

 $\text{ind} \leftarrow \text{ind} + 1$ **end for** $\text{minSpPath} \leftarrow$ a path corresponds to minimum value of StopPoints $\triangleright$ If there exist multiple paths, select the path with the minimum perimeter.display minSpPath including stop points

3.3 Angle-constrained MPP

In sections 3.1 and 3.2, the angle-constrained TSP and the angle-constrained MSP are introduced, respectively. The angle-constrained TSP tries to minimize the number of stop points to walk on the optimal TSP path and the angle-constrained MSP tries to minimize that to visit all vertices. In this section, we define the angle-constrained MPP as the problem of finding an angle-constrained path with the minimum perimeter.

Definition 5 Let S be a set of vertices in the plane and α an angle between 0 and π . The *angle-constrained MPP* is the problem of finding a path $CT = \langle x_1, x_2, \dots, x_{n+m}, x_1 \rangle$ with the minimum perimeter such that $\angle x_i \leq \alpha$ for all $i \in \{1, 2, \dots, n+m\}$ and $S \subseteq \{x_1, x_2, \dots, x_{n+m}\}$. We named CT , angle-constrained MPP path.

Here, we use the visible graph G which is defined in section 3.2 to solve the angle-constrained MPP on a set S of vertices. If there exist multiple paths from $v_{1,1}$ to $v_{n+1,n+1}$ on G , we find the path with the minimum perimeter. If there is no path, we will look for missing edges that satisfy the minimum perimeter by adding stop points to the path. Algorithm 7 computes the angle-constrained MPP path on a set S of vertices.

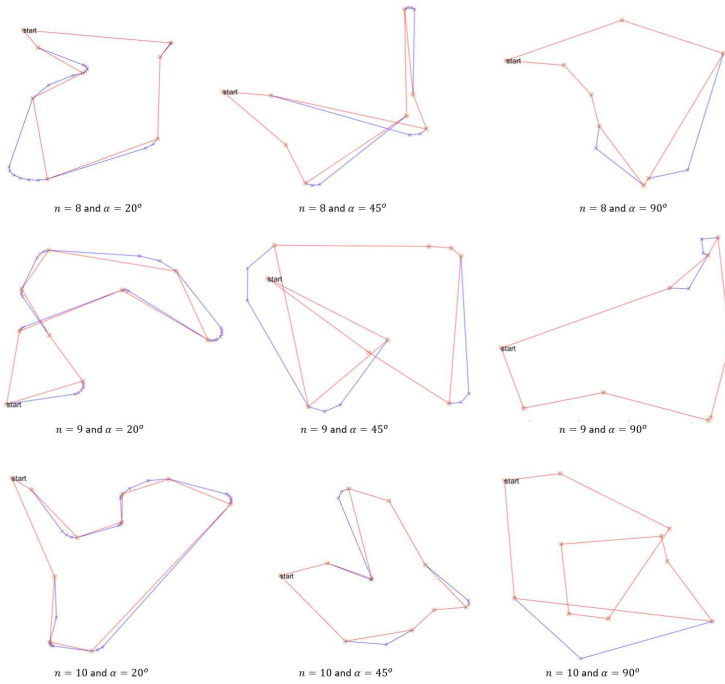


Fig. 11 Angle-constrained MSP paths for different values of α and different numbers of vertices

Fig. 12 shows angle-constrained MPP paths on a set of vertices for different values of α . In addition, Fig. 13 shows angle-constrained MSP, angle-constrained MPP, and angle-constrained TSP paths on a set of 10 points. In Fig. 13.a, the angle-constrained MSP path with 1 stop point is illustrated. As shown in Fig. 13.b, the angle-constrained MPP path has the minimum perimeter among all possible angle-constrained paths. The angle-constrained TSP path which is depicted in Fig. 13.c, shows an angle-constrained path to travel on the optimal TSP path.

4 Conclusion and future works

The major problem investigated in this paper is that of finding angle-restricted paths on a given set S of vertices in the plane. We presented an algorithm to compute a path with the minimum number of stop points to walk on the optimal TSP path, under the turn angle constraint. In addition, we presented two other algorithms to compute the angle-constrained paths with the minimum number of stop points and the minimum perimeter on S . We implemented the algorithms and examined them on several datasets.

Here, we defined three problems to compute angle-constrained paths on a set of vertices. It seems that all of these problems are NP-complete. Proving

Algorithm 7 Computing angle-constrained MPP path

Require: S, α **Ensure:** CT as a solution of angle-constrained MPP on S $n \leftarrow \text{size of } S$ $G \leftarrow$ visible graph computed by algorithm 5 using the parameters S and α $\text{minPath} \leftarrow$ find a path from $v_{1,1}$ to $v_{n+1,n+1}$ in G with the minimum perimeter.**if** minPath is not null **then** $\triangleright$ No stop point is needed display minPath

Exit

end if $\text{ind} \leftarrow 1$ **for all** $\text{path} \in$ all possible paths from $v_{1,1}$ to $v_{n+1,n+1}$ **do** $\text{missedEdges} \leftarrow$ all edges that are in path and are not in G . $\text{StopPoints}[\text{ind}] \leftarrow$ all required stop points for missedEdges . $\text{Perimeters}[\text{ind}] \leftarrow$ the perimeter of the entire path by adding $\text{StopPoints}[\text{ind}]$. $\text{ind} \leftarrow \text{ind} + 1$ **end for** $\text{minPrPath} \leftarrow$ a path corresponds to minimum value of Perimeters $\triangleright$

If there exist multiple paths, select the path with the minimum number of stop points.

display minPrPath including stop points

the NP-completeness of these problems for all values of α or a specific value of α , can be considered as future works. Moreover, presenting approximation algorithms to compute these paths is interesting as well.

Declarations

The authors declare that they have no conflict of interest.

Ethical Approval

The research meets all applicable standards with regard to the ethics of experimentation and research integrity, and the following is being certified/declared true.

As an expert scientist and along with co-authors of concerned field, the paper has been submitted with full responsibility, following due ethical procedure, and there is no duplicate publication, fraud, plagiarism, or concerns about animal or human experimentation.

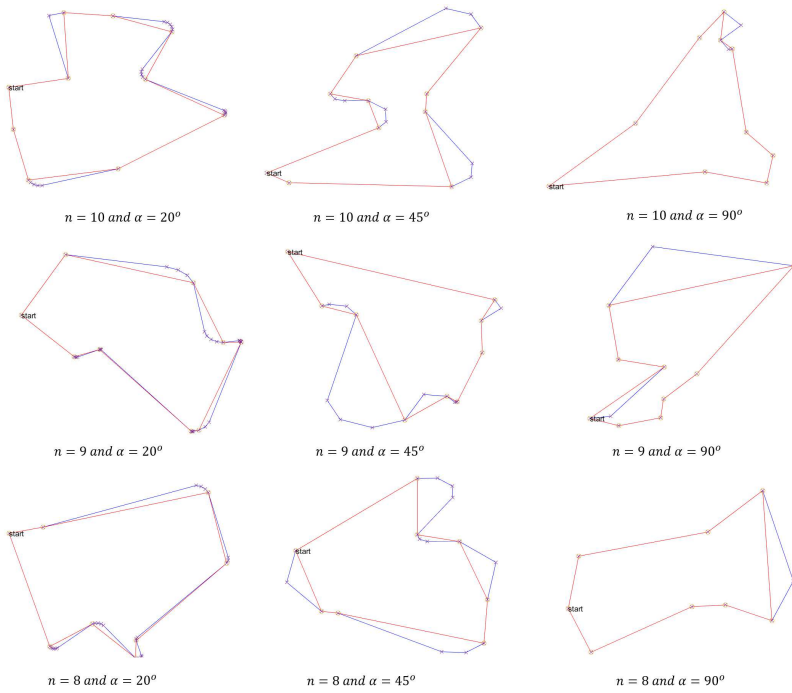


Fig. 12 Angle-constrained MPP paths for different values of α and different numbers of vertices

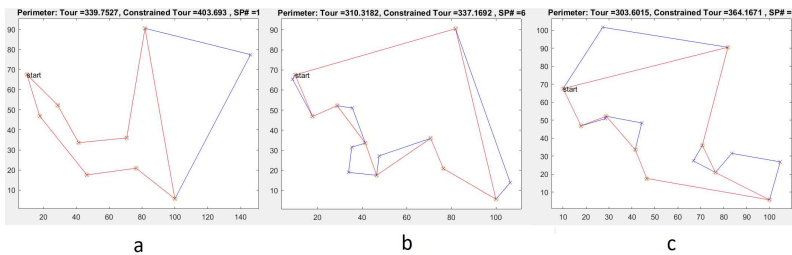


Fig. 13 Angle-constrained paths for $\alpha = 90^\circ$: a) angle-constrained MSP path with 1 stop point, b) angle-constrained MPP path with the minimum perimeter, and c) angle-constrained TSP path

Authors' contributions

All authors wrote the main manuscript text, prepared all figures, designed and implemented the presented algorithm and analyzed the results. All authors reviewed and approved the manuscript.

Funding

The research of the first and second authors are partially supported by the University of Kashan under grant numbers 991449/3 and 1143902/2, respectively.

Availability of data and materials

not applicable

References

- [1] K. Stoilova, T. Stoilov, Transportation modelling and solving travelling salesman problem, in: IOP Conference Series: Materials Science and Engineering, Vol. 878, IOP Publishing, 2020, p. 012026.
- [2] K. Suresh, V. Vaithyanathan, S. Venugopal, Solving travelling salesman problem and mapping to solve robot motion planning through genetic algorithm principle, *Indian Journal of Science and Technology* 8 (35).
- [3] J. E. Tito, M. E. Yacelga, M. C. Paredes, A. J. Utreras, W. Wójcik, O. Ussatova, Solution of travelling salesman problem applied to wireless sensor networks (wsn) through the mst and b&b methods, in: *Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments 2018*, Vol. 10808, International Society for Optics and Photonics, 2018, p. 108082F.
- [4] J. M. Wiener, N. N. Ehbauer, H. A. Mallot, Path planning and optimization in the traveling salesman problem: nearest neighbor vs. region-based strategies, in: *Dagstuhl Seminar Proceedings, Schloss Dagstuhl-Leibniz-Zentrum für Informatik*, 2007.
- [5] J. Chen, F. Ye, Y. Li, Travelling salesman problem for uav path planning with two parallel optimization algorithms, in: *2017 progress in electromagnetics research symposium-fall (PIERS-FALL)*, Ieee, 2017, pp. 832–837.
- [6] J. L. Santos, A. Oliveira, Traveling salesman problem in a geographic information management system, in: *Progress in Industrial Mathematics: Success Stories*, Springer, 2021, pp. 131–144.
- [7] G. Laporte, The traveling salesman problem: An overview of exact and approximate algorithms, *European Journal of Operational Research* 59 (2) (1992) 231–247.
- [8] M. Fischetti, A. Lodi, P. Toth, Exact methods for the asymmetric traveling salesman problem, in: *The traveling salesman problem and its variations*,

Springer, 2007, pp. 169–205.

- [9] J. Gromicho, J. Paixão, I. Bronco, Exact solution of multiple traveling salesman problems, in: Combinatorial optimization, Springer, 1992, pp. 291–292.
- [10] J.-Y. Potvin, Genetic algorithms for the traveling salesman problem, *Annals of Operations Research* 63 (3) (1996) 337–370.
- [11] L. M. Gambardella, M. Dorigo, Ant-q: A reinforcement learning approach to the traveling salesman problem, in: Machine learning proceedings 1995, Elsevier, 1995, pp. 252–260.
- [12] A. Rao, S. K. Hegde, A. Rao, K. Hegde, I. Rao, K. Hegde, A. Rao, S. Hegde, Literature survey on travelling salesman problem using genetic algorithms, *International Journal of Advanced Research in Education Technology (IJARET)* 2 (1) (2015) 42.
- [13] I. Brezina Jr, Z. Čičková, Solving the travelling salesman problem using the ant colony optimization, *Management Information Systems* 6 (4) (2011) 10–14.
- [14] L. E. Dubins, On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents, *American Journal of mathematics* 79 (3) (1957) 497–516.
- [15] J. Ny, E. Feron, E. Frazzoli, On the dubins traveling salesman problem, *IEEE Transactions on Automatic Control* 57 (1) (2011) 265–270.
- [16] P. Váňa, J. Faigl, On the dubins traveling salesman problem with neighborhoods, in: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2015, pp. 4029–4034.
- [17] Z. Chen, T. Shima, Shortest dubins paths through three points, *Automatica* 105 (2019) 368–375.
- [18] B. Jha, Z. Chen, T. Shima, On shortest dubins path via a circular boundary, *Automatica* 121 (2020) 109192.
- [19] P. Sujit, B. Hudzietz, S. Saripalli, Route planning for angle constrained terrain mapping using an unmanned aerial vehicle, *Journal of Intelligent & Robotic Systems* 69 (1) (2013) 273–283.
- [20] B. Triggs, Motion planning for nonholonomic vehicles: An introduction.